Unit - 3

1D Array in C

A one-dimensional (1D) array is a collection of elements of the same data type stored in contiguous memory locations. Each element is accessed using an index.

Declaration of 1D Array

Array declaration specifies the data type, array name, and number of elements.

Syntax: data_type array_name[size];

Example: int marks[5];

This declares an integer array named marks that can store five elements.

Array indexing:

Element	Index
First element	0
Second element	1
Third element	2
Fourth element	3
Fifth element	4

In C, array indexing starts from 0 and ends at size - 1.

Input in 1D Array

The scanf() function is used to take input for array elements.

Example:

```
int a[5], i;
for (i = 0; i < 5; i++)
{
   scanf("%d", &a[i]);
}
```

The loop takes input for all five elements, one by one.

Output of 1D Array

The printf() function is used to display array elements.

Example:

```
int a[5] = {10, 20, 30, 40, 50};
int i;
for (i = 0; i < 5; i++)
{
   printf("%d ", a[i]);
}
```

Output: 10 20 30 40 50

Traversal of 1D Array

Array traversal is the process of visiting and accessing each element of an array one by one.

A for loop is commonly used for traversal.

Syntax:

```
for (i = 0; i < size; i++)
{
   // Process array element
}
```

Example:

```
#include <stdio.h>
int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int i;
   for (i = 0; i < 5; i++)
   {
      printf("a[%d] = %d\n", i, a[i]);
   }
   return 0;
}
```

Output:

a[0] = 10

a[1] = 20

a[2] = 30

a[3] = 40

a[4] = 50

Complete Program: Input, Output and Traversal

```
#include <stdio.h>
int main()
{
   int a[5], i;
   printf("Enter 5 elements:\n");
   for (i = 0; i < 5; i++)
   {
      scanf("%d", &a[i]);
   }
   printf("Array elements are:\n");
   for (i = 0; i < 5; i++)
   {
      printf("%d ", a[i]);
   }
   return 0;
}
```

Output:

10 20 30 40 50

Array elements are:

10 20 30 40 50

1. Find Maximum Element in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 25, 5, 40, 15};

int i, max = a[0];

for (i = 1; i < 5; i++)

{

if (a[i] > max)

max = a[i];

}

printf("Maximum = %d", max);

return 0;

}
```

Output:

Maximum = 40

2. Find Minimum Element in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 25, 5, 40, 15};

int i, min = a[0];

for (i = 1; i < 5; i++)

{
```

```
if (a[i] < min)

min = a[i];

}

printf("Minimum = %d", min);

return 0;

}
```

Output:

Minimum = 5

3. Find Sum of Elements in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 20, 30, 40, 50};

int i, sum = 0;

for (i = 0; i < 5; i++)

{

sum = sum + a[i];

}

printf("Sum = %d", sum);

return 0;

}
```

Output:

Sum = 150

4. Find Average of Elements in a 1D Array

```
#include <stdio.h>

int main()

{

int a[5] = {10, 20, 30, 40, 50};

int i, sum = 0;

float avg;

for (i = 0; i < 5; i++)

{

sum = sum + a[i];

}

avg = (float)sum / 5;

printf("Average = %.2f", avg);

return 0;

}
```

Output:

Average = 30.00

ASCII values

In C, every character has a numerical value called an ASCII value. ASCII value is a number assigned to a character so that a computer can represent and process it.

ASCII values of characters

In C, every character has a numerical value called an ASCII value. In Program 19, these values are used to store the frequency of each character.

ASCII values of characters

Capital letters (A–Z)

Character	ASCII value
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Small letters (a–z)

Character	ASCII value
a	97
b	98
c	99
d	100
e	101
f	102
g	103
h	104
i	105
j	106
k	107
l	108
m	109
n	110
o	111
p	112
q	113
r	114
s	115
t	116
u	117
v	118
w	119
x	120
y	121
z	122

Digits (0–9)

Character	ASCII value
'0'	48
'1'	49
'2'	50
'3'	51
'4'	52
'5'	53
'6'	54
'7'	55
'8'	56
'9'	57

Program to print ASCII values

```
#include <stdio.h>

int main()

{

    char ch;

    printf("Enter a character: ");

    scanf("%c", &ch);

    printf("Character = %c\n", ch);

    printf("ASCII value = %d", ch);

    return 0;

}
```

Output:

```
Enter a character: A

Character = A

ASCII value = 65
```

Strings in C

A string in C is a sequence of characters terminated by the null character '\0'. Strings are stored in character arrays.

Example:

char str[20] = "Hello";

Header file required for string functions:

#include <string.h>

Example: -

```
#include <stdio.h>

int main()

{

   char str[100];

   printf("Enter a string: ");

   scanf("%s", str);

   printf("String = %s", str);

   return 0;

}
```

Output

Enter a string: Hello

String = Hello

In C, an array name automatically behaves like a pointer to its first element in most expressions. This is called array-to-pointer conversion.

1. Understand with an example

char str[5] = {'H', 'e', 'l', 'l', 'o'};

The array name str refers to the beginning of this array in most expressions.

- str points to the first character, H.
- &str[0] is the address of the first character.
- str[0] is the character H.

For a char array, str and &str[0] represent the same starting address when used in this context.

2. Why does scanf() need str, not &str?

scanf("%s", str);

scanf() needs the address where it should store the characters.

When you pass str, C automatically converts the array name to a pointer to its first element.

This is effectively equivalent to:

scanf("%s", &str[0]);

But &str is different: it represents the address of the entire array, with type char (*)[5] in this example, rather than a pointer to its first character.

3. Remember this rule

Expression	**Meaning**
str	Converts to a pointer to the first character in most expressions
str[0]	First character
&str[0]	Address of the first character
&str	Address of the entire array

In C, an array name is automatically converted into a pointer to its first element in most expressions. Therefore, scanf("%s", str) is correct because scanf() receives the address of the first character where it can store the input string.

1. gets() and puts()

gets() is used to read a string from the keyboard, including spaces. However, it is unsafe because it does not check the array size and has been removed from the C11 standard. Use fgets() instead.

Syntax:

fgets(str, sizeof(str), stdin);

puts() is used to display a string on the screen and automatically adds a newline.

Syntax:

puts(str);

Example:

```
#include <stdio.h>

int main()

{

   char str[50];

   printf("Enter a string: ");

   fgets(str, sizeof(str), stdin);

   puts(str);

   return 0;

}
```

Output:

Enter a string: Hello World

Hello World

Program: Read and Print a String with Spaces

```
#include <stdio.h>

int main()

{

   char str[100];
```

```
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    printf("String = %s", str);
    return 0;
}
```

Output:

Enter a string: Hello World

String = Hello World

2. strlen()

The strlen() function returns the length of a string, excluding the null character '\0'.

Syntax: strlen(string);

Example:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[] = "Hello";
    printf("Length = %d", (int)strlen(str));
    return 0;
}
```

Output:

Length = 5

3. strcpy()

The strcpy() function copies the source string into the destination string, including the terminating null character.

Syntax:

strcpy(destination, source);

Example:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char source[] = "Hello";
    char destination[20];
    strcpy(destination, source);
    printf("%s", destination);
    return 0;
}
```

Output:

Hello

Program: Copy a String Without Using strcpy()

```
#include <stdio.h>
int main()
{
    char str1[100], str2[100];
    int i = 0;
    printf("Enter a string: ");
    scanf("%s", str1);
    while (str1[i] != '\0')
```

```
    {
        str2[i] = str1[i];
        i++;
    }
    str2[i] = '\0';
    printf("Copied string = %s", str2);
    return 0;
}
```

Output:

Enter a string: Hello

Copied string = Hello

Explanation:

- The loop copies each character of str1 into str2.
- The loop stops when it reaches '\0'.
- str2[i] = '\0' adds the string terminator to the copied string.
- str2 can then be printed using %s.

4. strcmp()

The strcmp() function compares two strings lexicographically, character by character.

Syntax:

strcmp(string1, string2);

Return values:

- 0 — both strings are equal.
- Negative value — the first string is smaller.
- Positive value — the first string is greater.

Example:

```
#include <stdio.h>
#include <string.h>
int main() {
   char str1[100], str2[100];
   int result;
   printf("Enter first string: ");
   scanf("%s", str1);
   printf("Enter second string: ");
   scanf("%s", str2);
   result = strcmp(str1, str2);
   if (result == 0)
      printf("Strings are equal");
   else if (result < 0)
      printf("%s is smaller than %s", str1, str2);
   else
      printf("%s is greater than %s", str1, str2);
   return 0;
}
```

Output

```
Enter first string: android
Enter second string: ankit
android is smaller than ankit
```

Explanation

The strcmp() function compares two strings character by character using their ASCII values.

Position	First string	ASCII	Second string	ASCII
1	a	97	a	97
2	n	110	n	110
3	d	100	k	107

1). Position 1: a = 97 and a = 97. Both characters are equal, so comparison continues.

2). Position 2: n = 110 and n = 110. Both characters are equal, so comparison continues.

3). Position 3: d = 100 and k = 107. The characters are different.

Since 100 < 107, the comparison result is negative. Therefore, android is smaller than ankit.

Important: As soon as strcmp() finds the first different characters, it determines the result and stops comparing further characters. It does not compare the remaining characters.

For example, it does not need to compare roid with it because the difference has already been found at position 3.

- result == 0 → Strings are equal.
- result < 0 → First string is smaller.
- result > 0 → First string is greater.

Difference Between String Functions

Function	Purpose
fgets()	Reads a string safely with a specified size limit.
puts()	Displays a string followed by a newline.
strlen()	Calculates the length of a string.
strcpy()	Copies one string into another.
strcmp()	Compares two strings.

Program: Concatenate Two Strings Using strcat()

```
#include <stdio.h>

#include <string.h>

int main()

{

   char str1[200], str2[100];

   printf("Enter first string: ");

   scanf("%s", str1);

   printf("Enter second string: ");

   scanf("%s", str2);

   strcat(str1, str2);

   printf("Concatenated string = %s", str1);

   return 0;

}
```

Output:

```
Enter first string: Hello

Enter second string: World

Concatenated string = HelloWorld
```

Explanation:

- Concatenation means joining two strings.
- strcat(str1, str2) appends str2 to the end of str1.
- The destination array str1 must have enough space for both strings and the terminating '\0'.

Program: Count Alphabets, Digits and Special Characters

```
#include <stdio.h>

int main()

{

   char str[100];

   int i = 0, alphabets = 0, digits = 0, special = 0;

   printf("Enter a string: ");

   scanf("%s", str);

   while (str[i] != '\0')

   {

      if ((str[i] >= 'A' && str[i] <= 'Z') ||

         (str[i] >= 'a' && str[i] <= 'z'))

         alphabets++;

      else if (str[i] >= '0' && str[i] <= '9')

         digits++;

      else

         special++;


      i++;

   }

   printf("Alphabets = %d\n", alphabets);

   printf("Digits = %d\n", digits);

   printf("Special characters = %d", special);

   return 0;
```

}

Output:

Enter a string: Abc123@

Alphabets = 3

Digits = 3

Special characters = 1

Explanation:

- The first condition checks whether a character is an uppercase or lowercase English letter.
- The second condition checks whether it is a digit from 0 to 9.
- Any remaining character is counted as a special character.
- Because scanf("%s", str) stops at whitespace, spaces are not included in this input.

Pointer in C

A pointer is a variable that stores the memory address of another variable. It allows us to access and modify the value of another variable indirectly using its address.

For example, if an integer variable a contains the value 10 and its memory address is 1000, a pointer variable p can store the address 1000. Using the pointer, we can access the value 10 stored at that address.

Address of a Variable

An address is the location of a variable in computer memory. Whenever a variable is created, memory is allocated to store its value.

The & operator, called the address-of operator, is used to obtain the address of a variable.

Example:

#include <stdio.h>

int main()

{

```
    int a = 10;
    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    return 0;
}
```

Output:

Value of a = 10

Address of a = 0x7ffd1234

The address shown is an example. The actual address depends on the program's execution environment.

Explanation:

- int a = 10; creates an integer variable named a and stores 10 in it.
- a represents the value stored in the variable, which is 10.
- &a gives the memory address of variable a.
- %d is used to print an integer value.
- %p is used to print a memory address.

Pointer Variable

A pointer variable is a variable that stores the memory address of another variable.

Syntax

data_type *pointer_name;

Example:

int *p;

Here:

- int indicates that p can point to an integer variable.
- * indicates that p is declared as a pointer.

- p is the name of the pointer variable.

A pointer must be assigned a valid address before it is dereferenced using *.

Program Using a Pointer

```
#include <stdio.h>

int main()

{

   int a = 10;

   int *p;

   p = &a;

   printf("Value of a = %d\n", a);

   printf("Address of a = %p\n", &a);

   printf("Address stored in p = %p\n", p);

   printf("Value using pointer = %d\n", *p);

   return 0;

}
```

Output:

Value of a = 10

Address of a = 0x7ffd1234

Address stored in p = 0x7ffd1234

Value using pointer = 10

The memory addresses in the output are illustrative and may differ when the program runs.

Detailed Explanation of the Program

Consider these statements:

int a = 10;

int *p;

p = &a;

Step 1: int a = 10;

An integer variable a is created, and the value 10 is stored in it.

Suppose its memory address is 1000.

Step 2: int *p;

A pointer variable named p is declared. It can store the address of an integer variable.

At this point, p has not been initialized, so it must not be used to access a value yet.

Step 3: p = &a;

The expression &a gives the address of variable a.

That address is assigned to p. If the address of a is 1000, then p stores 1000.

Step 4: printf("%d\n", a);

This prints the value stored directly in a, which is 10.

Step 5: printf("%p\n", &a);

This prints the memory address of a.

Step 6: printf("%p\n", p);

This prints the address stored in pointer p. Since p stores the address of a, it prints the same address as &a.

Step 7: printf("%d\n", *p);

The expression *p accesses the value at the memory address stored in p.

Since p points to a, *p gives 10.

Working of a Pointer in Memory

Suppose the memory address of a is 1000.

Variable a

Value = 10

Address = 1000

p = &a

Pointer stores the address 1000

Pointer variable p

Stored address = 1000

*p accesses the value 10 stored at address 1000.

This example shows the difference between a variable's value, its address, and the address stored in a pointer.

Difference Between a, &a, p, and *p

Assume a = 10 and the address of a is 1000.

Expression	**Meaning**	**Result**
a	Value of variable a	10
&a	Address of variable a	1000 (example)
p	Address stored in pointer p	1000 (example)
*p	Value at the address stored in p	10

Remember:

- a gives the value.
- &a gives the address.
- p gives the address stored in the pointer.
- *p gives the value at that address.

Important Pointer Operators

Operator	**Name**	**Purpose**
&	Address-of operator	Obtains the address of a variable

*	Indirection (dereference) operator	Declares a pointer or accesses the value at the stored address

The meaning of * depends on its use:

int *p; // Declares a pointer

*p = 20; // Changes the value at the address stored in p

For the second statement, p must already point to a valid, writable integer variable.

Modifying a Variable Using a Pointer

A pointer can be used to modify the value of the variable it points to.

Program:

```
#include <stdio.h>

int main()

{

   int a = 10;

   int *p = &a;

   printf("Before modification = %d\n", a);

   *p = 20;

   printf("After modification = %d\n", a);

   return 0;

}
```

Output:

Before modification = 10

After modification = 20

Explanation:

- int *p = &a; stores the address of a in p.

- *p = 20; changes the value stored at that address to 20.
- Because p points to a, the value of a also becomes 20.

Advantages of Pointers

1). **Indirect access:** Pointers allow access to variables through their memory addresses.

2). **Function arguments:** They allow functions to modify variables belonging to the calling function.

3). **Dynamic memory allocation:** Pointers are used with functions such as malloc() and calloc().

4). **Data structures:** They are essential for structures such as linked lists, trees, and graphs.

5). **Arrays and strings:** They help access and process array elements and strings efficiently.

Structures in C

A structure in C is a user-defined data type that groups variables of different data types under a single name.

It is used to store related information of different types in one unit.

For example, a student has a roll number (int), a name (char array) and a percentage (float). These variables can be grouped together using a structure.

Syntax of Structure

```
struct structure_name
{
   data_type member1;
   data_type member2;
   data_type member3;
};
```

Explanation:

- struct is the keyword used to define a structure.

- structure_name is the name given to the structure.
- member1, member2, and member3 are the structure members.
- Each member can have a different data type.
- The semicolon (;) after the closing brace is compulsory.

Example:

```
struct Student
{
   int roll;
   char name[30];
   float marks;
};
```

Here, Student is the structure name, and roll, name, and marks are its members.

Declaration of Structure Variables

After defining a structure, variables can be declared to store its data.

Method 1: Declare variables after defining the structure

```
struct Student
{
   int roll;
   char name[30];
   float marks;
};
struct Student s1, s2;
```

Here, s1 and s2 are structure variables of type struct Student.

Method 2: Declare variables while defining the structure

```
struct Student
{
   int roll;
   char name[30];
   float marks;
} s1, s2;
```

Both methods are valid in C.

Accessing Structure Members

Structure members are accessed using the dot (.) operator.

Syntax:

```
structure_variable.member_name;
```

Example:

```
s1.roll
s1.name
s1.marks
```

Here:

- s1.roll accesses the roll number.
- s1.name accesses the student's name.
- s1.marks accesses the student's marks.

Complete Program Using Structure

```
#include <stdio.h>
struct Student
{
```

```
    int roll;
    char name[30];
    float marks;
};
int main()
{
    struct Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.roll);
    printf("Enter name: ");
    scanf("%s", s1.name);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);
    printf("\nStudent Details\n");
    printf("Roll Number = %d\n", s1.roll);
    printf("Name = %s\n", s1.name);
    printf("Marks = %.2f\n", s1.marks);
    return 0;
}
```

Output:

```
Enter roll number: 101
Enter name: Ankit
Enter marks: 85.5
Student Details
```

Roll Number = 101

Name = Ankit

Marks = 85.50

Explanation of the Program

1). struct Student defines a structure with three members: roll, name, and marks.

2). struct Student s1; declares a structure variable named s1.

3). scanf("%d", &s1.roll); reads the roll number.

4). scanf("%s", s1.name); reads the student's name.

5). scanf("%f", &s1.marks); reads the marks.

6). The dot operator (.) is used to access individual members.

7). printf() displays the stored information.

Important: The & operator is used with s1.roll and s1.marks in scanf(). It is not required with s1.name because the character array name provides the starting address of the array.

Initialization of Structure

Structure members can be initialized when a structure variable is declared.

Example:

```
#include <stdio.h>

struct Student

{

   int roll;

   char name[30];

   float marks;

};

int main()

{
```

```
    struct Student s1 = {101, "Ankit", 85.5};
    printf("Roll Number = %d\n", s1.roll);
    printf("Name = %s\n", s1.name);
    printf("Marks = %.2f\n", s1.marks);
    return 0;
}
```

Output:

```
Roll Number = 101
Name = Ankit
Marks = 85.50
```

The values are assigned to members in the order in which the members are declared in the structure.

Here:

- 101 is assigned to roll.
- "Ankit" is assigned to name.
- 85.5 is assigned to marks.

Array of Structures

An array of structures is a collection of structure variables of the same structure type. It is used to store records of multiple students, employees, or other entities.

Syntax:

```
struct Student s[5];
```

This declares an array of five structure variables. Each element can store one student's information.

Program:

```
#include <stdio.h>
struct Student
{
   int roll;
   char name[30];
   float marks;
};
int main()
{
   struct Student s[2];
   int i;
   for (i = 0; i < 2; i++)
   {
      printf("Enter roll, name and marks: ");
      scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);
   }
   printf("\nStudent Details\n");
   for (i = 0; i < 2; i++) {
      printf("%d %s %.2f\n",
            s[i].roll, s[i].name, s[i].marks);
   }
   return 0;
}
```

Output:

Enter roll, name and marks: 101 Ankit 85.5

Enter roll, name and marks: 102 Rahul 90

Student Details

101 Ankit 85.50

102 Rahul 90.00

Explanation:

- struct Student s[2]; creates an array of two student records.
- s[i].roll accesses the roll number of the ith student.
- s[i].name accesses the name of the ith student.
- s[i].marks accesses the marks of the ith student.
- The loops input and display the records.

Nested Structure

A nested structure is a structure that contains another structure as one of its members.

It is useful when a record contains another related record, such as a student's date of birth or address.

Program:

```
#include <stdio.h>

struct Date

{

   int day;

   int month;

   int year;

};

struct Student
```

```
{
    int roll;
    char name[30];
    struct Date dob;
};
int main()
{
    struct Student s1 = {101, "Ankit", {1, 1, 2000}};
    printf("Roll Number = %d\n", s1.roll);
    printf("Name = %s\n", s1.name);
    printf("Date of Birth = %02d-%02d-%d\n",
           s1.dob.day, s1.dob.month, s1.dob.year);
    return 0;
}
```

Output:

```
Roll Number = 101
Name = Ankit
Date of Birth = 01-01-2000
```

Explanation:

- struct Date contains day, month, and year.
- struct Student contains a member named dob of type struct Date.
- s1.dob.day accesses the day.
- s1.dob.month accesses the month.
- s1.dob.year accesses the year.

The dot operator is used repeatedly to access members of a nested structure.

Structure vs Array

Structure	**Array**
Groups related members, which can have different data types.	Stores elements of the same data type.
Members are accessed by their names using the dot operator.	Elements are accessed using index numbers.
Example: s1.roll	Example: a[0]
Used to represent records such as students and employees.	Used to store collections of similar values.

Advantages of Structures

1). Groups related data: Stores different types of information under one name.

2). Organizes records: Makes student, employee, and product records easier to manage.

3). Supports arrays of records: Allows multiple records of the same structure type to be stored together.

4). Supports nested data: One structure can contain another structure.

5). Works with pointers and functions: Structures can be accessed through pointers and passed to functions.

6). Improves program readability: Related variables are grouped into a meaningful unit.